

Authored by: Mani Kalra, ONYEWUCHI KENNEDY EBEGBULEM

Duration: 180 mins

Level: Intermediate

Pre-requisite Skills: Python, Data Analysis, Pandas, Time-Series Forecasting, Deep Learning, Data Visualisation

Scenario

Urban Infrastructure Demand Forecasting

Melbourne is experiencing rapid urban growth, which places significant pressure on its essential infrastructure services such as water, electricity, and transport. Without accurate predictions of demand, cities face challenges in managing resources, planning for future growth, and preventing service disruptions. Current urban planning often lacks unified forecasting models that can integrate diverse data sources—like population forecasts, floor space projections, and environmental resource consumption—to provide a holistic view of future infrastructure needs. This makes it challenging to strategically allocate resources and develop resilient urban systems.

Project Objective

The objective of this project is to develop and implement advanced forecasting models to predict the demand for essential urban infrastructure services across various city regions in Melbourne. By integrating diverse urban datasets and leveraging time-series and deep learning methodologies, the project aims to provide actionable insights for urban planning, resource allocation, and sustainable city development.

Project Stages

Stage 1: Data Collection & Integration Collect and integrate multi-source datasets from the City of Melbourne open data portal and other relevant sources, including population forecasts, floor space projections, development activity, environmental resource consumption, and quality-of-life indicators. Preprocess data by validating structures, aligning time granularities, handling missing values, and standardising formats for unified analysis.

Stage 2: Exploratory Data Analysis (EDA) Analyze data distributions, relationships between variables, and identify potential patterns or anomalies. Visualize time-series trends, spatial concentrations, and demographic shifts across different datasets to understand their individual characteristics and interdependencies.

Stage 3: Data Preprocessing and Feature Engineering Clean datasets by handling missing values through imputation or removal. Convert data types as necessary (e.g., date, categorical features). Engineer new features from raw data to enhance model performance, such as creating time-based features (e.g., year, month, day of week) and aggregating data to appropriate temporal granularities.

Stage 4: Time-Series Forecasting Model Development Develop and implement advanced time-series forecasting models. This includes:

- **Classical Models:** Explore traditional time-series methods (e.g., ARIMA, Exponential Smoothing) for baseline predictions.
- **Deep Learning Models:** Focus on Long Short-Term Memory (LSTM) networks and conceptualize the application of Temporal Fusion Transformers (TFTs) for multi-horizon forecasting, leveraging their ability to handle complex temporal dynamics and multi-variate inputs.

Stage 5: Model Evaluation and Validation Evaluate model performance using appropriate metrics (e.g., MAE, RMSE, MAPE) for time-series forecasting. Validate models against unseen data to ensure generalizability and reliability of predictions. Discuss limitations, especially concerning data sparsity.

Stage 6: Visualisation and Reporting Generate visualisations of forecasts alongside actual data to illustrate model performance and trends. Summarise key findings, technical progress, and provide recommendations for urban planning and resource management based on the forecasting results. Document the project's methodology, achievements, and insights in a comprehensive report.

Expected Outcomes

- **Accurate Demand Forecasts:** Deliver reliable predictions for water, electricity, and transport demand across Melbourne's regions.
- **Actionable Insights:** Provide data-driven recommendations for strategic urban planning, infrastructure development, and resource allocation.
- **Enhanced Decision-Making:** Equip city planners and policymakers with tools to anticipate future needs and mitigate potential service disruptions.
- **Scalable Methodology:** Establish a flexible and expandable framework for continuous monitoring and refinement of infrastructure demand forecasting.

Resources

Here are the datasets used in this project:

1. **City of Melbourne Floor Space Forecasts by Small Area (2020-2040)**
Contains projections for commercial and residential floor space across different small areas in Melbourne. This dataset is crucial for understanding future space utilisation and its impact on infrastructure demand. Link: [Floor Space Forecasts](#)
2. **City of Melbourne Population Forecasts by Small Area (2020-2040)**
Provides detailed population forecasts by small area, gender, and age group. Essential for predicting future demographic changes that directly influence infrastructure usage. Link: [Population Forecasts](#)
3. **Development Activity Monitor** Monitors current and completed development projects in Melbourne, including details on residential dwellings, commercial floor space, and construction years. This dataset helps in assessing new infrastructure demands generated by urban development. Link: [Development Activity Monitor](#)
4. **Environmental Resources Consumed (Electricity, Fuel, Water, etc.)** Data on various environmental resources consumed, including electricity, fuel, and water. This provides historical consumption patterns vital for forecasting future demand for utility services. Link: [Environmental Resources Consumed](#)
5. **Indicators of Quality of Life and City Services by Year** Includes various indicators related to quality of life and city services, which can serve as covariates influencing infrastructure demand or as proxies for urban health and development. Link: [Quality of Life Indicators](#)
6. **Melbourne Energy Consumption with Suburbs Geo-coding Done** A processed dataset containing energy consumption information for Melbourne properties, enriched with suburb-level geo-coding. This is critical for understanding spatial energy demand patterns. Link: [Melbourne Energy Consumption](#)

✓  Urban Planning

✓ Read data using API

For this project, we utilised a selection of datasets that are most relevant to forecasting urban infrastructure demand. These datasets include population

forecasts, floor space projections, development activity, environmental resource consumption, and quality-of-life indicators for the City of Melbourne.

During the data preparation phase, we observed that some available datasets were either incomplete or contained significant missing values. To maintain the accuracy and reliability of our analysis, such datasets were excluded from the study. Only datasets with sufficient completeness and relevance were selected to ensure meaningful insights and robust forecasting outcomes. The following cells will read the data from the provided API links into pandas DataFrames.

```
import pandas as pd

file_path = 'https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data'
df_floor_space = pd.read_csv(file_path)

print(f"Successfully loaded: {file_path}")
display(df_floor_space.head())
```

Successfully loaded: <https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data>

	Geography	Year	Category	Industry	Space Use	Value
0	Docklands	2028	Residential floorspace by space use		house	60279
1	Docklands	2043	Residential floorspace by space use		house	60279
2	East Melbourne	2026	Residential floorspace by space use		apartment	287771
-						

```
import pandas as pd

file_path = 'https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data'
df_population_forecasts = pd.read_csv(file_path)

print(f"Successfully loaded: {file_path}")
display(df_population_forecasts.head())
```

Successfully loaded: <https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data>

	Geography	Year	Gender	Age	Value
0	Kensington	2037	Male	Age 45-49	747
1	Kensington	2038	Female	Age 20-24	428
2	Kensington	2038	Female	Age 75-79	144
3	Kensington	2038	Male	Age 50-54	267
4	Kensington	2038	Male	Age 75-79	172

```
import pandas as pd
```

```
file_path = 'https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data'
df_development_activity = pd.read_csv(file_path)
```

```
print(f"Successfully loaded: {file_path}")
display(df_development_activity.head())
```

Successfully loaded: <https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data>

	data_format	development_key	status	year_completed	clue_small_area
--	-------------	-----------------	--------	----------------	-----------------

0	Pre May 16	X000459	COMPLETED	2005.0	North Melbourne
---	------------	---------	-----------	--------	-----------------

1	Pre May 16	X000573	COMPLETED	2013.0	West Melbourne (Residential)
---	------------	---------	-----------	--------	---------------------------------

2	Pre May 16	X000448	COMPLETED	2015.0	North Melbourne
---	------------	---------	-----------	--------	-----------------

3	Pre May 16	X000449	COMPLETED	2009.0	North Melbourne
---	------------	---------	-----------	--------	-----------------

4	Pre May 16	X000394	COMPLETED	2002.0	Carlton
---	------------	---------	-----------	--------	---------

5 rows × 42 columns

```
import pandas as pd
```

```
file_path = 'https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data'
df_environmental_resources = pd.read_csv(file_path)
```

```
print(f"Successfully loaded: {file_path}")
display(df_environmental_resources.head())
```

Successfully loaded: <https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/entry/1>

	Branch	Asset Name	Resource Category	Resource Type	Amount	Unit	Total Greenhouse Gases (t CO2-e)	Billi Sta da
0	Property Services	Toilets - Flinders Street Station	Water	Water	81.629630	kL	111.016296	20106-
1	Property Services	Toilets - Queen Street	Water	Water	44.800549	kL	60.928746	20106-
2	Property Services	Lease - Multicultural Hub	Water	Water	72.844216	kL	99.068133	20106-
		Lease - Property Aged Care						201

```
import pandas as pd
```

```
file_path = 'https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/data'
df_quality_of_life = pd.read_csv(file_path)
```

```
print(f"Successfully loaded: {file_path}")
display(df_quality_of_life.head())
```

Successfully loaded: <https://data.melbourne.vic.gov.au/api/explore/v2.1/catalog/entry/1>

	Number	Indicator Theme	Indicator Variable	Indicator Descriptor	Type	Greater Melbourne (Value)	Greater Melbourne (Year)
0	5.4	Economy	Full-time Employment	Percentage of persons in fulltime employment	Supporting	50.6319	2014.0
1	5.7	Economy	Number of New Patents	Number of new patents per 100 000 population p...	Supporting	13.6251	2014.0
2	6.3	Education	Secondary Education Completion	Percentage of students completing secondary ed...	Core	91.8848	2013.0
3	7.1	Energy	Residential Electrical	Total residential electrical	Core	NaN	NaN

```
import pandas as pd
```

```
file_path = 'https://raw.githubusercontent.com/Chameleon-company/MOP-Code/main/data/energy_consumption.csv'
df_energy_consumption = pd.read_csv(file_path)
```

```
##### haved uploaded a new file with location names defined instead, as geo
print(f"Successfully loaded: {file_path}")
display(df_energy_consumption.head())
```

Successfully loaded: <https://raw.githubusercontent.com/Chameleon-company/MOP->

	Geo Point	Suburb	Latitude	Longitude	Geo Shape
0	-37.83363154012436, 144.98420174077557	Richmond	-37.833632	144.984202	{ "coordinates": [[[-37.833632, 144.984202], [-37.833632, 144.98447264054937], [-37.833632, 144.98447264054937], [-37.833632, 144.984202]]]
1	-37.79900382289352, 144.93494112057527	West Melbourne	-37.799004	144.934941	{ "coordinates": [[[-37.799004, 144.934941], [-37.799004, 144.93552203684675], [-37.799004, 144.93552203684675], [-37.799004, 144.934941]]]
2	-37.800632838889626, 144.9488946131469	North Melbourne	-37.800633	144.948895	{ "coordinates": [[[-37.800633, 144.948895], [-37.800633, 144.94882546176322], [-37.800633, 144.94882546176322], [-37.800633, 144.948895]]]
3	-37.801031454537835, 144.95223929995998	North Melbourne	-37.801031	144.952239	{ "coordinates": [[[-37.801031, 144.952239], [-37.801031, 144.95222140005745], [-37.801031, 144.95222140005745], [-37.801031, 144.952239]]]
4	-37.8004776165655, 144.94154189092782	West Melbourne	-37.800478	144.941542	{ "coordinates": [[[-37.800478, 144.941542], [-37.800478, 144.9418606474646], [-37.800478, 144.9418606474646], [-37.800478, 144.941542]]]

Objective 1: Forecasting Infrastructure Usage - Starting with Energy Consumption

To understand and forecast infrastructure usage, it's crucial to look at how demand changes over time. Let's begin by preparing and visualising the

`df_energy_consumption` data. This dataset has consumption values for specific years (e.g., 2011, 2016, 2021) spread across different columns, which isn't ideal for time-series analysis. We'll reshape it into a more usable format where each row represents a specific year's consumption for a given property.

```
import pandas as pd

# Dictionary of all loaded dataframes, including the original ones
all_dataframes = {
    'df_floor_space': df_floor_space,
    'df_population_forecasts': df_population_forecasts,
    'df_development_activity': df_development_activity,
    'df_environmental_resources': df_environmental_resources,
    'df_quality_of_life': df_quality_of_life,
    'df_energy_consumption': df_energy_consumption
}

# List to keep track of processed dataframes
```

```

processed_dataframes = {}

print("Starting date column standardization and initial inspection...")

for df_name, df in all_dataframes.items():
    df_copy = df.copy() # Work on a copy to avoid modifying original DFs
    print(f"\n--- Processing {df_name} ---")

    # Identify potential date/year columns and convert them
    if 'Year' in df_copy.columns:
        # For dataframes with a 'Year' column, convert it to datetime object
        df_copy['Date'] = pd.to_datetime(df_copy['Year'], format='%Y')
        print(f"Converted 'Year' column to 'Date' (datetime) in {df_name}.")
    elif 'year_completed' in df_copy.columns:
        # For df_development_activity, 'year_completed' can be a date refere
        df_copy['Date'] = pd.to_datetime(df_copy['year_completed'], format='%Y')
        print(f"Converted 'year_completed' to 'Date' (datetime) in {df_name}")
        # Drop rows where 'year_completed' couldn't be coerced to a valid da
        df_copy.dropna(subset=['Date'], inplace=True)
    elif 'Billing Start date' in df_copy.columns:
        # For environmental resources, use 'Billing Start date'
        df_copy['Date'] = pd.to_datetime(df_copy['Billing Start date'], erro
        print(f"Converted 'Billing Start date' to 'Date' (datetime) in {df_n
        df_copy.dropna(subset=['Date'], inplace=True)
    elif 'Melbourne municipality (Year)' in df_copy.columns:
        # For quality of life, use 'Melbourne municipality (Year)'
        df_copy['Date'] = pd.to_datetime(df_copy['Melbourne municipality (Ye
        print(f"Converted 'Melbourne municipality (Year)' to 'Date' (datetim
        df_copy.dropna(subset=['Date'], inplace=True)
    elif 'total_2011' in df_copy.columns and 'total_2016' in df_copy.columns:
        # For df_energy_consumption, this looks like a snapshot rather than
        # We might need to reshape this later, but for now, just acknowledge
        print(f"{df_name} contains year-specific columns like 'total_2011',
        if 'Suburb' in df_copy.columns:
            df_copy['Location'] = df_copy['Suburb']
            print(f"Added 'Location' column from 'Suburb' in {df_name}.")
        else:
            print(f"No 'Suburb' column found in {df_name}. 'Location' column
    else:
        print(f"No obvious date/year column found for direct conversion in {

    # Display info and first few rows of the processed dataframe
    print(df_copy.info())
    display(df_copy.head())

    processed_dataframes[df_name + '_processed'] = df_copy

print("\nDate standardization and initial inspection complete.")

```


Starting date column standardization and initial inspection...

--- Processing df_floor_space ---

Converted 'Year' column to 'Date' (datetime) in df_floor_space.

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9345 entries, 0 to 9344

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Geography	9345 non-null	object
1	Year	9345 non-null	int64
2	Category	9345 non-null	object
3	Industry Space Use	9345 non-null	object
4	Value	9345 non-null	int64
5	Date	9345 non-null	datetime64[ns]

dtypes: datetime64[ns](1), int64(2), object(3)

memory usage: 438.2+ KB

None

	Geography	Year	Category	Industry Space Use	Value	Date
0	Carlton	2023	Residential floorspace by space use	apartment	680226	2023-01-01
1	Carlton	2038	Residential floorspace by space use	house	231953	2038-01-01
2	Docklands	2028	Residential floorspace by space use	apartment	975903	2028-01-01
3	Docklands	2023	Residential floorspace by space use	house	60279	2023-01-01
4	Docklands	2026	Residential floorspace by space use	house	60279	2026-01-01

--- Processing df_population_forecasts ---

Converted 'Year' column to 'Date' (datetime) in df_population_forecasts.

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10584 entries, 0 to 10583

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Geography	10584 non-null	object
1	Year	10584 non-null	int64
2	Gender	10584 non-null	object
3	Age	10584 non-null	object
4	Value	10584 non-null	int64
5	Date	10584 non-null	datetime64[ns]

dtypes: datetime64[ns](1), int64(2), object(3)

memory usage: 496.3+ KB

None

	Geography	Year	Gender	Age	Value	Date
0	Kensington	2037	Male	Age 45-49	747	2037-01-01
1	Kensington	2038	Female	Age 20-24	428	2038-01-01

```

import pandas as pd

# Get the processed dataframe for energy consumption
df_energy_processed = processed_dataframes['df_energy_consumption_processed']

print("Columns in df_energy_processed:")
print(df_energy_processed.columns)

# Identify columns that represent energy consumption for different years
year_columns = [col for col in df_energy_processed.columns if 'total_' in col]

# Reshape the dataframe using `melt` to convert year columns into rows
df_energy_long = df_energy_processed.melt(id_vars=['Geo Point', 'Location'],
                                           value_vars=year_columns,
                                           var_name='Year_Str',
                                           value_name='Total_Energy_Consumpti')

# Extract the year from the 'Year_Str' column and convert to datetime
df_energy_long['Year'] = df_energy_long['Year_Str'].str.replace('total_', '')
df_energy_long['Date'] = pd.to_datetime(df_energy_long['Year'], format='%Y')

# Drop the original 'Year_Str' column as we now have 'Year' and 'Date'
df_energy_long = df_energy_long.drop(columns=['Year_Str'])

print("Reshaped df_energy_consumption_processed into a time-series friendly")
display(df_energy_long.head())
print(df_energy_long.info())

```

19	student_beds	1110	non-null	int64
20	student_accommodation_units	1110	non-null	int64
21	institutional_accom_beds	1110	non-null	int64
22	hotel_rooms	1110	non-null	int64
23	serviced_apartments	1110	non-null	int64
24	hotels_serviced_apartments	1110	non-null	int64
25	hostel_beds	1110	non-null	int64
26	childcare_places	1110	non-null	int64
27	office_flr	1110	non-null	int64
28	retail_flr	1110	non-null	int64
29	industrial_flr	1110	non-null	int64
30	storage_flr	1110	non-null	int64
31	education_flr	1110	non-null	int64
32	hospital_flr	1110	non-null	int64
33	recreation_flr	1110	non-null	int64
34	publicdispaly_flr	1110	non-null	int64
35	community_flr	1110	non-null	int64
36	car_spaces	1110	non-null	int64
37	bike_spaces	1110	non-null	int64
38	town_planning_application	1110	non-null	object
39	longitude	1110	non-null	float64
40	latitude	1110	non-null	float64
41	geopoint	1110	non-null	object
42	Date	1110	non-null	datetime64[ns]

dtypes: datetime64[ns](1), float64(7), int64(28), object(7)
 memory usage: 381.6+ KB
 None

data_format	development_key	status	year_completed	clue_small_ar
-------------	-----------------	--------	----------------	---------------

```

Columns in df_energy_processed:
Index(['Geo Point', 'Suburb', 'Latitude', 'Longitude', 'Geo Shape',
      'total_2011', 'p_com_2011', 'p_res_2026', 'p_com_2026', 'total_2026',
      'p_com_2021', 'property_id', 'p_res_2011', 'total_2016', 'total_2021',
      'p_res_2021', 'floor_area', 'p_res_2016', 'p_com_2016', 'Location'],
      dtype='object')
1 Reshaped df_energy_consumption_processed into a time-series friendly format.

```

	Geo Point	Location	Total_Energy_Consumption_kwh	Year	Date
0	-37.83363154012436, 144.98420174077557	Richmond	15.836556	2011	2011-01-01
1	-37.79900382289352, 144.93494112057527	West Melbourne	338.761383	2011	2011-01-01
3	Pre May 16	X000449 COMPLETED	2009.0	2009	2009-01-01
2	-37.800632838889626, 144.9488946131469	North Melbourne	19.007585	2011	2011-01-01
3	-37.801031454537835, 144.9523392995998	North Melbourne	2.376923	2011	2011-01-01
4	-37.8004776165655, 144.94154189092782	West Melbourne	531.058950	2011	2011-01-01

```

RangeIndex: 9474 entries, 0 to 9473
Data columns (total 13 columns):
#0   Branch
#1   Geo Point
#2   Location
#3   Total_Energy_Consumption_kwh
#4   Year
#5   Date
#6   total_2011
#7   p_com_2011
#8   p_res_2026
#9   p_com_2026
#10  total_2026
#11  p_com_2021
#12  p_res_2011

```

```

Data columns (total 5 columns):
#0   Branch
#1   Geo Point
#2   Location
#3   Total_Energy_Consumption_kwh
#4   Year

```

```

Non-Null Count  Dtype

```

```

0   Branch      9474 non-null  object

```

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get the processed dataframe for energy consumption
# df_energy_processed is already updated with 'Location' column from the pre

print("Columns in df_energy_processed:")
print(df_energy_processed.columns)

# Identify columns that represent energy consumption for different years
year_columns = [col for col in df_energy_processed.columns if 'total_' in col]

# Reshape the dataframe using `melt` to convert year columns into rows
# Now include 'Location' along with 'Geo Point' as id_vars
df_energy_long = df_energy_processed.melt(id_vars=['Geo Point', 'Location'],
                                           value_vars=year_columns,
                                           var_name='Year_Str',
                                           value_name='Total_Energy_Consumption_kwh')

# Extract the year from the 'Year_Str' column and convert to datetime
df_energy_long['Year'] = df_energy_long['Year_Str'].str.replace('total_', '')
df_energy_long['Date'] = pd.to_datetime(df_energy_long['Year'], format='%Y')

```

Greater	Greater
---------	---------

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get the processed dataframe for energy consumption
df_energy_processed = processed_dataframes['df_energy_consumption_processed']

print("Columns in df_energy_processed:")
print(df_energy_processed.columns)

# Identify columns that represent energy consumption for different years
year_columns = [col for col in df_energy_processed.columns if 'total_' in col]

# Reshape the dataframe using `melt` to convert year columns into rows
# Now include 'Location' along with 'Geo Point' as id vars
```

```

# Columns: df_energy_processed: Dtype
Index(['Geo Point', 'Suburb', 'Latitude', 'Longitude', 'Geo Shape',
0  total_2011', 'p_com_2011', 'p_res_2026', 'p_com_2026', 'total_2026',
1  'p_res_2021', 'p_com_2021', 'property_id', 'p_res_2011', 'total_2016', 'total_2021',
2  'p_res_2021', 'p_com_2021', 'floor_area', 'p_res_2016', 'p_com_2016', 'Location'],
3  dtype=object, non-null float64
4  Geo Shape energy consumption processed into a time-series friendly format,
5  total_2011 13587 non-null float64
6  p_com_2011 13587 non-null float64

Reshaped into a time-series friendly format,
0  -37.83363154012436, Richmond 15.836556 2011 2011-01-01
11  property_id 13587 non-null float64
12  p_res_2021 13587 non-null float64
13  total_2016 13587 non-null float64
14  total_2021 13587 non-null float64

2  -37.800632838889626, North 19.007585 2011 2011-01-01
144.9488946131469 Melbourne
18  p_com_2016 13587 non-null float64
19  Location 13587 non-null object
3  -37.801031454537835, North 2.376923 2011 2011-01-01
144.95223929993998 Melbourne
memory usage: 2.1+ MB

4  -37.8004776165655, West 531.058950 2011 2011-01-01
144.94154189092782 Melbourne

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54348 entries, 0 to 54347
Data columns (total 5 columns):

```

Now that our energy consumption data is in a proper time-series format, let's visualise the overall trend. We'll aggregate the total consumption across all properties for each available year to see how it has changed.

Energy Consumption by Building Type

To get a deeper understanding, let's also visualise how different building types contribute to the total energy consumption and if their trends vary. This can highlight specific areas of infrastructure demand.

Double-click (or enter) to edit

Start coding or [generate](#) with AI.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get the processed dataframe for energy consumption
df_energy_processed = processed_dataframes['df_energy_consumption_processed']

print("Columns in df_energy_processed:")
print(df_energy_processed.columns)

# Identify columns that represent energy consumption for different years
year_columns = [col for col in df_energy_processed.columns if 'total_' in col]

# Reshape the dataframe using `melt` to convert year columns into rows
df_energy_long = df_energy_processed.melt(id_vars=['Geo Point', 'Location'],
                                           value_vars=year_columns,
                                           var_name='Year_Str',
                                           value_name='Total_Energy_Consumption')

# Extract the year from the 'Year_Str' column and convert to datetime
df_energy_long['Year'] = df_energy_long['Year_Str'].str.replace('total_', '')
df_energy_long['Date'] = pd.to_datetime(df_energy_long['Year'], format='%Y')

# Drop the original 'Year_Str' column as we now have 'Year' and 'Date'
df_energy_long = df_energy_long.drop(columns=['Year_Str'])

print("Reshaped df_energy_consumption_processed into a time-series friendly")
display(df_energy_long.head())
print(df_energy_long.info())
```

```
Columns in df_energy_processed:
Index(['Geo Point', 'Suburb', 'Latitude', 'Longitude', 'Geo Shape',
      'total_2011', 'p_com_2011', 'p_res_2026', 'p_com_2026', 'total_2026',
      'p_com_2021', 'property_id', 'p_res_2011', 'total_2016', 'total_2021',
      'p_res_2021', 'floor_area', 'p_res_2016', 'p_com_2016', 'Location'],
      dtype='object')
Reshaped df_energy_consumption_processed into a time-series friendly format.
```

	Geo Point	Location	Total_Energy_Consumption_kWh	Year	Date
0	-37.83363154012436, 144.98420174077557	Richmond	15.836556	2011	2011-01-01
1	-37.79900382289352, 144.93494112057527	West Melbourne	338.761383	2011	2011-01-01
2	-37.800632838889626, 144.9488946131469	North Melbourne	19.007585	2011	2011-01-01
3	-37.801031454537835, 144.95223929995998	North Melbourne	2.376923	2011	2011-01-01
4	-37.8004776165655, 144.94154189092782	West Melbourne	531.058950	2011	2011-01-01

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54348 entries, 0 to 54347
Data columns (total 5 columns):
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get the processed dataframe for energy consumption
df_energy_processed = processed_dataframes['df_energy_consumption_processed']

print("Columns in df_energy_processed:")
print(df_energy_processed.columns)

# Identify columns that represent energy consumption for different years
year_columns = [col for col in df_energy_processed.columns if 'total_' in col]

# Reshape the dataframe using `melt` to convert year columns into rows
# Now include 'Location' along with 'Geo Point' as id_vars
df_energy_long = df_energy_processed.melt(id_vars=['Geo Point', 'Location'],
                                           value_vars=year_columns,
                                           var_name='Year_Str',
                                           value_name='Total_Energy_Consumption_kWh')

# Extract the year from the 'Year_Str' column and convert to datetime
df_energy_long['Year'] = df_energy_long['Year_Str'].str.replace('total_', '')
df_energy_long['Date'] = pd.to_datetime(df_energy_long['Year'], format='%Y')
```



```
# Drop the original 'Year_Str' column as we now have 'Year' and 'Date'
df_energy_long = df_energy_long.drop(columns=['Year_Str'])

print("Reshaped df_energy_consumption_processed into a time-series friendly
display(df_energy_long.head())
print(df_energy_long.info())
```

Columns in df_energy_processed:
Index(['Geo Point', 'Suburb', 'Latitude', 'Longitude', 'Geo Shape',
'total_2011', 'p_com_2011', 'p_res_2026', 'p_com_2026', 'total_2026',
'p_com_2021', 'property_id', 'p_res_2011', 'total_2016', 'total_2021',
'p_res_2021', 'floor_area', 'p_res_2016', 'p_com_2016', 'Location'],
dtype='object')

Reshaped df_energy_consumption_processed into a time-series friendly format,

	Geo Point	Location	Total_Energy_Consumption_kWh	Year	Date
0	-37.83363154012436, 144.98420174077557	Richmond	15.836556	2011	2011-01-01
1	-37.79900382289352, 144.93494112057527	West Melbourne	338.761383	2011	2011-01-01
2	-37.800632838889626, 144.9488946131469	North Melbourne	19.007585	2011	2011-01-01
3	-37.801031454537835, 144.95223929995998	North Melbourne	2.376923	2011	2011-01-01
4	-37.8004776165655, 144.94154189092782	West Melbourne	531.058950	2011	2011-01-01

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54348 entries, 0 to 54347
Data columns (total 5 columns):
```

Visualising Total Energy Consumption Over Time by Location

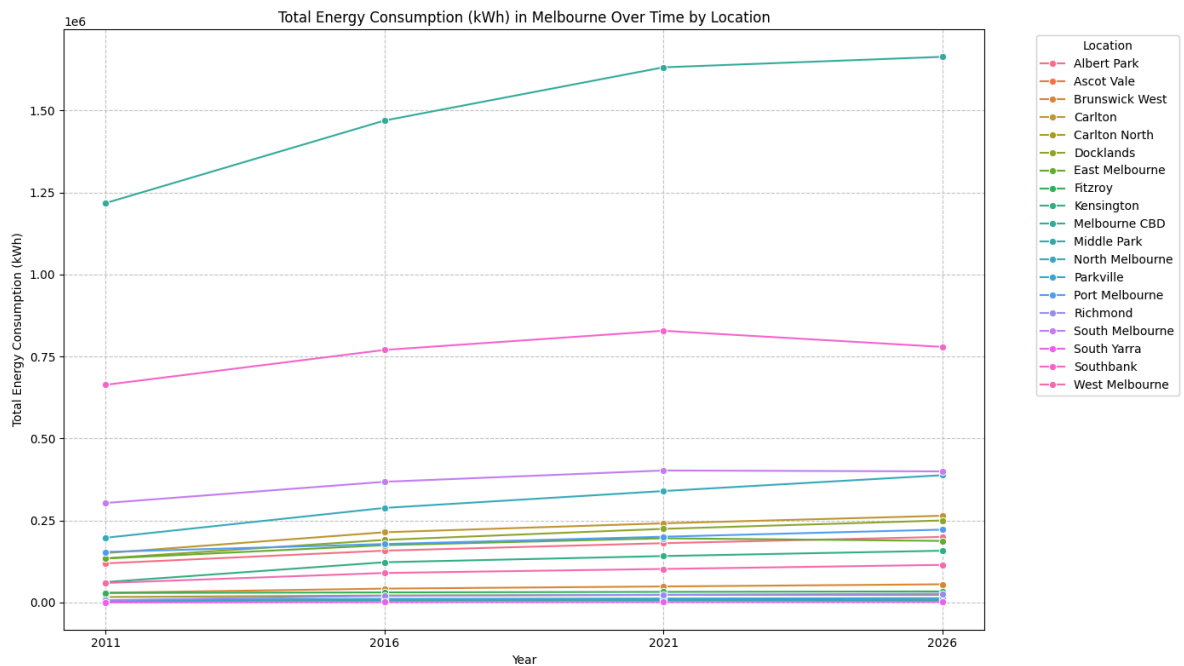
Now that our energy consumption data includes human-readable locations and is in a proper time-series format, let's visualise the overall trend. We'll aggregate the total consumption across all properties for each available year, and also break it down by the newly identified 'Location' to see how it has changed in different areas.

```
# Group by Year and Location and sum the total energy consumption
annual_total_energy_by_location = df_energy_long.groupby(['Year', 'Location'])

plt.figure(figsize=(14, 8))
sns.lineplot(x='Year', y='Total_Energy_Consumption_kWh', hue='Location', data=
plt.title('Total Energy Consumption (kWh) in Melbourne Over Time by Location
plt.xlabel('Year')
plt.ylabel('Total Energy Consumption (kWh)')
```



```
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(annual_total_energy_by_location['Year'].unique())
plt.legend(title='Location', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



Energy Consumption by Building Type and Location

To get an even deeper understanding, let's also visualise how different building types contribute to the total energy consumption within each specific human-readable

location, and if their trends vary. This can highlight specific areas and building types that are driving infrastructure demand.

Deriving Building Type Information from Energy Consumption Data

Since the `df_environmental_resources` does not contain the building type information we're looking for, we will extract 'Commercial' and 'Residential' building types directly from our `df_energy_processed` dataframe. This involves a more detailed reshaping to separate the consumption values based on their type prefixes (`p_com_` and `p_res_`).

```
import pandas as pd

# Get the processed dataframe for energy consumption
df_energy_processed = processed_dataframes['df_energy_consumption_processed']

# Identify columns for commercial and residential consumption
com_year_columns = [col for col in df_energy_processed.columns if 'p_com_' in col]
res_year_columns = [col for col in df_energy_processed.columns if 'p_res_' in col]

# Melt for Commercial consumption
df_com_long = df_energy_processed.melt(id_vars=['Geo Point', 'property_id',
                                              'value_vars=com_year_columns',
                                              'var_name='Year_Str',
                                              'value_name='Energy_Consumption_kWh'])
df_com_long['Building Type'] = 'Commercial'
df_com_long['Year'] = df_com_long['Year_Str'].str.replace('p_com_', '').astype(int)

# Melt for Residential consumption
df_res_long = df_energy_processed.melt(id_vars=['Geo Point', 'property_id',
                                              'value_vars=res_year_columns',
                                              'var_name='Year_Str',
                                              'value_name='Energy_Consumption_kWh'])
df_res_long['Building Type'] = 'Residential'
df_res_long['Year'] = df_res_long['Year_Str'].str.replace('p_res_', '').astype(int)

# Concatenate the commercial and residential dataframes
df_energy_long_typed = pd.concat([df_com_long, df_res_long], ignore_index=True)

# Convert 'Year' to datetime
df_energy_long_typed['Date'] = pd.to_datetime(df_energy_long_typed['Year'],
                                              format='%Y')

# Drop the original 'Year_Str' column
df_energy_long_typed = df_energy_long_typed.drop(columns=['Year_Str'])

print("Created df_energy_long_typed with Commercial and Residential building types")
display(df_energy_long_typed.head())
print(df_energy_long_typed.info())
```

Created df_energy_long_typed with Commercial and Residential building types.

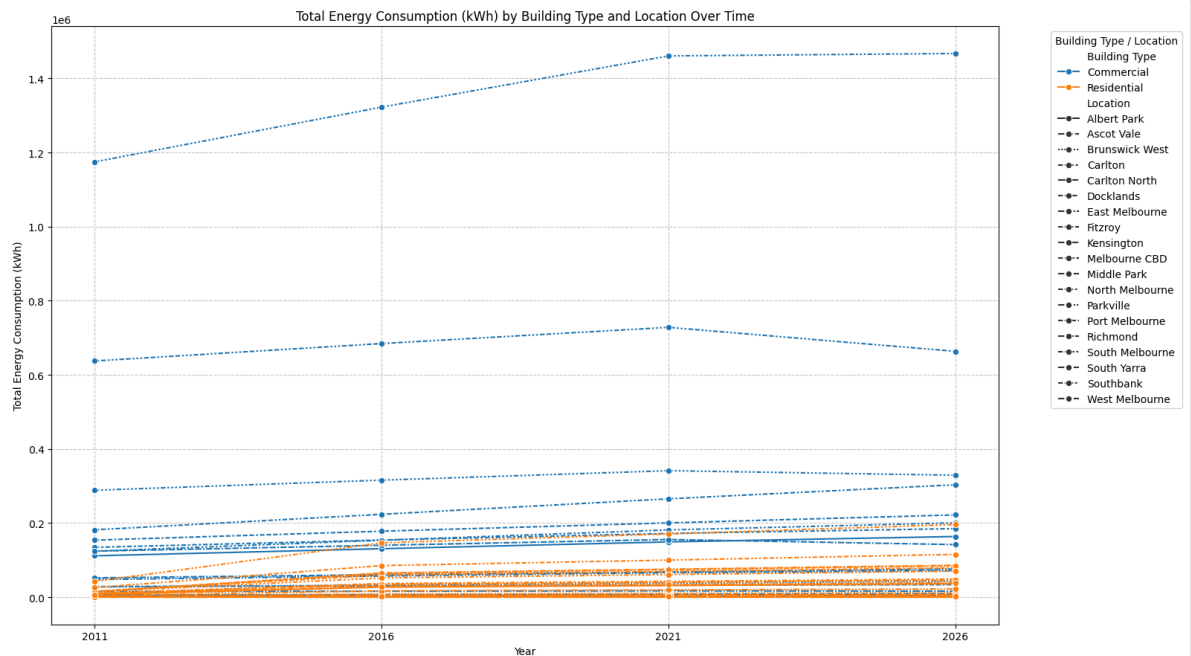
	Geo Point	property_id	floor_area	Location	Energy_Consumption
0	-37.83363154012436, 144.98420174077557	100431.0	762.96	Richmond	0.0
1	-37.79900382289352, 144.93494112057527	100508.0	0.00	West Melbourne	338.7
2	-37.800632838889626, 144.9488946131469	100521.0	286.40	North Melbourne	0.0
3	-37.801031454537835, 144.95223929995998	100547.0	99.80	North Melbourne	0.0
4	-37.8004776165655, 144.94154189092782	100550.0	0.00	West Melbourne	531.0

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 108696 entries, 0 to 108695  
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----

```
energy_by_building_type_location = df_energy_long_typed.groupby(['Year', 'Bu
```

```
plt.figure(figsize=(16, 9))  
sns.lineplot(x='Year', y='Energy_Consumption_kWh', hue='Building Type', styl  
plt.title('Total Energy Consumption (kWh) by Building Type and Location Over  
plt.xlabel('Year')  
plt.ylabel('Total Energy Consumption (kWh)')  
plt.grid(True, linestyle='--', alpha=0.7)  
plt.xticks(energy_by_building_type_location['Year'].unique())  
plt.legend(title='Building Type / Location', bbox_to_anchor=(1.05, 1), loc=''  
plt.tight_layout()  
plt.show()
```

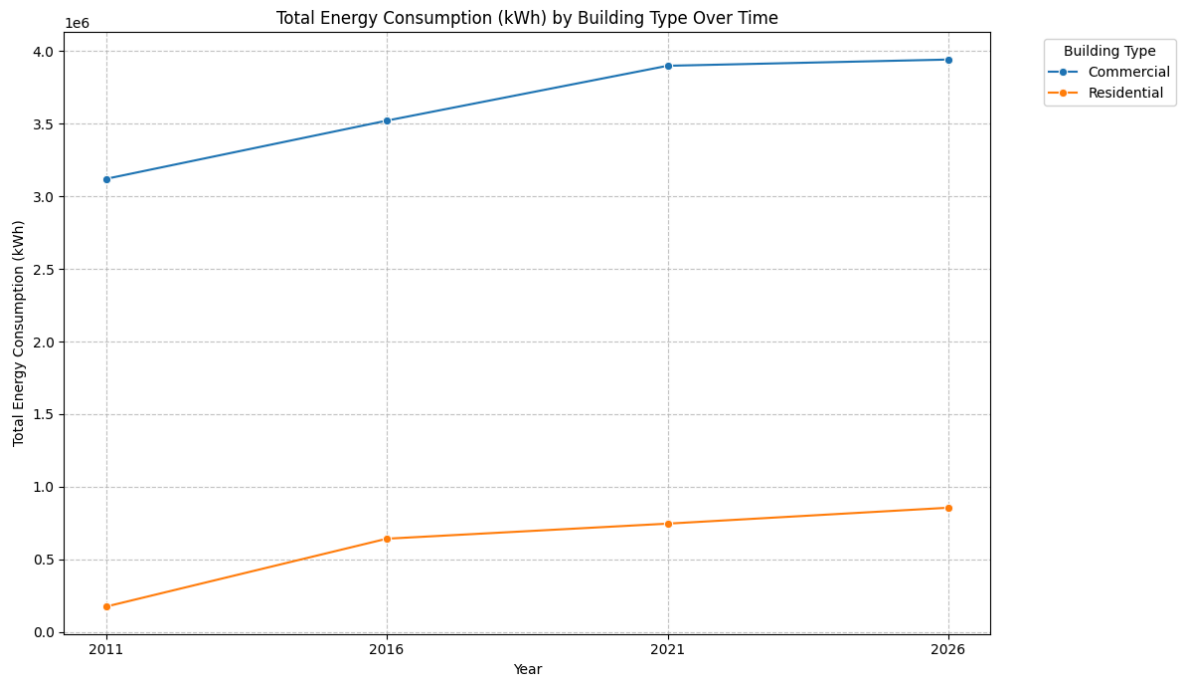


```
import matplotlib.pyplot as plt
import seaborn as sns

# Group by Year and Building Type and sum the total energy consumption
energy_by_building_type = df_energy_long_typed.groupby(['Year', 'Building Ty

plt.figure(figsize=(12, 7))
sns.lineplot(x='Year', y='Energy_Consumption_kWh', hue='Building Type', data
plt.title('Total Energy Consumption (kWh) by Building Type Over Time')
plt.xlabel('Year')
plt.ylabel('Total Energy Consumption (kWh)')
```

```
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(energy_by_building_type['Year'].unique())
plt.legend(title='Building Type', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



Visualising Energy Consumption by Building Type Over Time

Now that we have explicitly separated commercial and residential consumption, let's visualise how each building type contributes to the total energy demand over the years. This will provide valuable insights into infrastructure planning.

Forecasting Total Energy Consumption using LSTM

We will now attempt to forecast total energy consumption using a Long Short-Term Memory (LSTM) neural network. LSTMs are well-suited for time series forecasting tasks.

Important Note on Data Limitations: The `annual_total_energy` dataset, which we will use for this forecast, contains only 4 data points (representing total energy consumption for years 2011, 2016, 2021, and 2026). Training a deep learning model like an LSTM effectively requires significantly more data to learn underlying patterns and avoid overfitting. Therefore, the model's performance will be highly constrained by this limited data, and the forecasts should be interpreted with extreme caution and primarily as a demonstration of the methodology rather than a reliable prediction.

We will proceed with the following steps:

- Prepare the data by scaling and creating sequences.
- Define a simple LSTM model.
- Train the model.
- Generate and visualise predictions.

Checking for Missing Values Across All Processed Dataframes

Before we proceed with feature engineering and model development, it's crucial to examine the completeness of our datasets. Identifying and understanding missing values will help us decide on appropriate imputation strategies or data cleaning steps.

```
print("Checking for missing values in each processed dataframe...")

for df_name, df in processed_dataframes.items():
    print(f"\n--- Missing values in {df_name} ---")
    missing_values = df.isnull().sum()
    # Filter to show only columns with missing values and sort them
    missing_values = missing_values[missing_values > 0].sort_values(ascending=True)

    if not missing_values.empty:
        print("Found missing values in the following columns:")
        display(missing_values.to_frame(name='Missing Count'))
        print(f"Total missing values in {df_name}: {missing_values.sum()}")
    else:
        print("No missing values found in this dataframe.")

print("\nMissing value check complete.")
```

Checking for missing values in each processed dataframe...

--- Missing values in df_floor_space_processed ---
No missing values found in this dataframe.

--- Missing values in df_population_forecasts_processed ---
No missing values found in this dataframe.

--- Missing values in df_development_activity_processed ---
Found missing values in the following columns:

Missing Count	
property_id_5	1110
property_id_4	1103
property_id_3	1092
property_id_2	1020

Total missing values in df_development_activity_processed: 4325

--- Missing values in df_environmental_resources_processed ---
Found missing values in the following columns:

Missing Count	
Off-peak amount	7597
Peak amount	7462
Amount	6

Total missing values in df_environmental_resources_processed: 15065

--- Missing values in df_quality_of_life_processed ---
Found missing values in the following columns:

Missing Count	
Melbourne municipality (Units)	87
Greater Melbourne (Value)	15
Greater Melbourne (Year)	15
Greater Melbourne (Units)	2

Total missing values in df_quality_of_life_processed: 119

--- Missing values in df_energy_consumption_processed ---
No missing values found in this dataframe.

Missing value check complete.

Double-click (or enter) to edit

Analysing and Visualising Trends in Population Forecasts

Following the request to explore other datasets and move towards advanced forecasting models, we will now analyse `df_population_forecasts_processed`. This dataset provides population forecasts by year, gender, and age group, making it an excellent candidate for time-series analysis. We'll start by visualising overall population trends and then discuss how this data could be prepared for models like Temporal Fusion Transformers or Hybrid Spatiotemporal models.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure the 'Date' column is in datetime format and 'Value' is numeric
df_population = processed_dataframes['df_population_forecasts_processed'].copy()
df_population['Date'] = pd.to_datetime(df_population['Year'], format='%Y')
df_population['Value'] = pd.to_numeric(df_population['Value'])

print("Population DataFrame info:")
df_population.info()

# 1. Visualize Overall Population Trend
overall_population_trend = df_population.groupby('Date')['Value'].sum().reset_index()

plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Value', data=overall_population_trend, marker='o')
plt.title('Overall Population Forecast Trend in Melbourne (2023-2040)')
plt.xlabel('Year')
plt.ylabel('Total Population')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 2. Visualize Population Trend by Gender
population_by_gender = df_population.groupby(['Date', 'Gender'])['Value'].sum().reset_index()

plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Value', hue='Gender', data=population_by_gender, marker='o')
plt.title('Population Forecast Trend by Gender in Melbourne (2023-2040)')
plt.xlabel('Year')
plt.ylabel('Population Count')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(rotation=45)
plt.legend(title='Gender')
plt.tight_layout()
plt.show()

# 3. Visualize Population Trend by Age Group (Top N for clarity)
# Get top N age groups based on latest year's population
latest_year = df_population['Year'].max()
top_age_groups = df_population[df_population['Year'] == latest_year].groupby('Age').sum().reset_index()
population_by_age = df_population[df_population['Age'].isin(top_age_groups)]

plt.figure(figsize=(14, 7))
```



```
sns.lineplot(x='Date', y='Value', hue='Age', data=population_by_age, marker=
plt.title('Top 5 Population Forecast Trends by Age Group in Melbourne (2023-
plt.xlabel('Year')
plt.ylabel('Population Count')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(rotation=45)
plt.legend(title='Age Group', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

Population DataFrame info:

```
<class 'pandas.core.frame.DataFrame'>
```

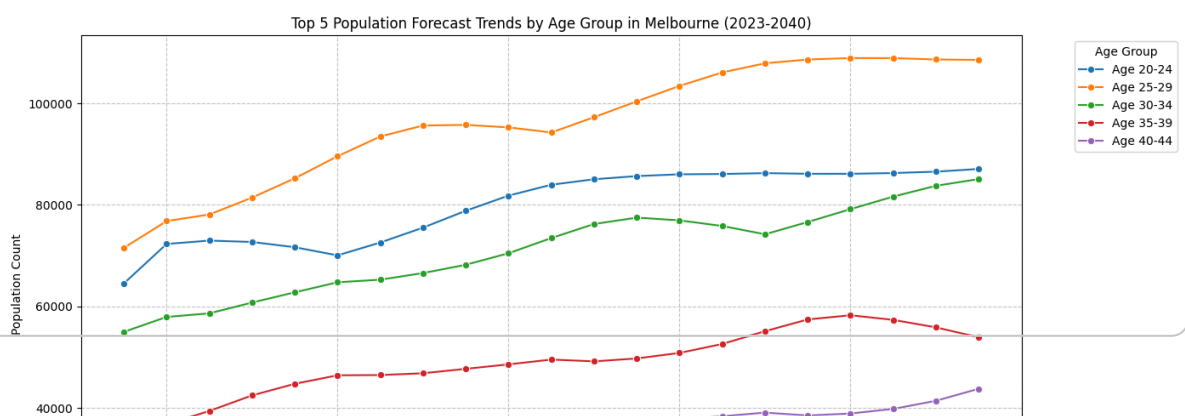
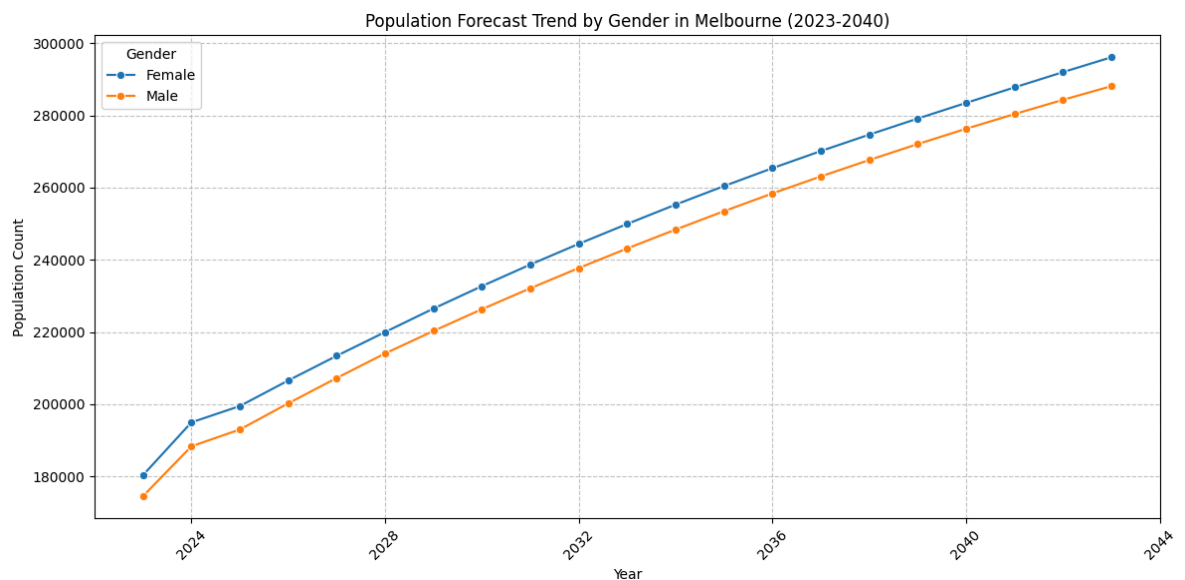
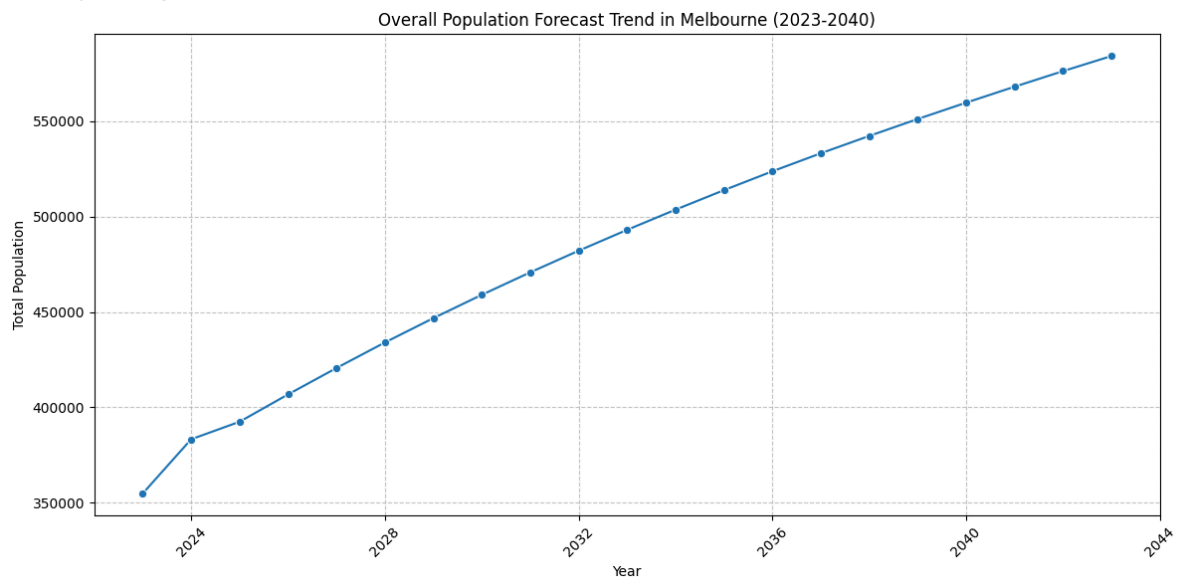
RangeIndex: 10584 entries, 0 to 10583

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Geography	10584 non-null	object
1	Year	10584 non-null	int64
2	Gender	10584 non-null	object
3	Age	10584 non-null	object
4	Value	10584 non-null	int64
5	Date	10584 non-null	datetime64[ns]

dtypes: datetime64[ns](1), int64(2), object(3)

memory usage: 496.3+ KB



Conceptual Data Structuring for Temporal Fusion

Transformers (TFT)

Temporal Fusion Transformers (TFTs) are a powerful deep learning architecture for multi-horizon time series forecasting. They are designed to handle complex temporal dynamics, including multi-variate inputs, static (time-invariant) covariates, and dynamic (time-varying) covariates. Key requirements for TFTs include:

- **Time Index:** A clear time variable (e.g., `Date`).
- **Target Variable:** The variable to be forecasted (e.g., `Value` for population).
- **Grouping Variables (Entities):** Variables that define individual time series (e.g., `Geography`, `Gender`, `Age`). A TFT can forecast multiple series simultaneously.
- **Static Covariates:** Features that do not change over time for a given entity (e.g., `Geography` might be treated as static if its characteristics are constant for the forecast horizon).
- **Dynamic Covariates:** Features that change over time, both known in the future (e.g., `Year`, `Month` if doing sub-annual forecasts) and unknown in the future (e.g., economic indicators).

For `df_population_forecasts_processed`, we could structure the data as follows:

- **Time Index:** `Date`
- **Target:** `Value`
- **Grouping Variables:** `Geography`, `Gender`, `Age` (as each combination defines a unique series)
- **Static Covariates:** `Geography` (as an embedding)
- **Dynamic Covariates (known in future):** `Year` (derived from `Date`). Other potential features like `Gender` and `Age` can also be considered dynamic if their influence is modelled as time-varying.

Libraries like `pytorch_forecasting` provide excellent tools to prepare data into the specific `TimeSeriesDataSet` format required by TFTs. This involves defining all the above components and handling features like embeddings for categorical variables and scaling for numerical ones. A full implementation would involve defining the `TimeSeriesDataSet`, then instantiating and training a `TemporalFusionTransformer` model.

```
print("Checking for missing values in each processed dataframe...")

for df_name, df in processed_dataframes.items():
    print(f"\n--- Missing values in {df_name} ---")
    missing_values = df.isnull().sum()
    # Filter to show only columns with missing values and sort them
    missing_values = missing_values[missing_values > 0].sort_values(ascending=True)
```

```
if not missing_values.empty:
    print("Found missing values in the following columns:")
    display(missing_values.to_frame(name='Missing Count'))
    print(f"Total missing values in {df_name}: {missing_values.sum()}")
else:
    print("No missing values found in this dataframe.")

print("\nMissing value check complete.")
```

Checking for missing values in each processed dataframe...

--- Missing values in df_floor_space_processed ---

No missing values found in this dataframe.

--- Missing values in df_population_forecasts_processed ---

No missing values found in this dataframe.

--- Missing values in df_development_activity_processed ---

Found missing values in the following columns:

	Missing Count
property_id_5	1110
property_id_4	1103
property_id_3	1092
property_id_2	1020

Total missing values in df_development_activity_processed: 4325

--- Missing values in df_environmental_resources_processed ---

Found missing values in the following columns:

	Missing Count
Off-peak amount	7597
Peak amount	7462
Amount	6

Total missing values in df_environmental_resources_processed: 15065

--- Missing values in df_quality_of_life_processed ---

Found missing values in the following columns:

	Missing Count
Melbourne municipality (Units)	87
Greater Melbourne (Value)	15
Greater Melbourne (Year)	15
Greater Melbourne (Units)	2

Total missing values in df_quality_of_life_processed: 119

--- Missing values in df_energy_consumption_processed ---

No missing values found in this dataframe.

Missing value check complete.

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Extract the energy consumption values
data = annual_total_energy['Total_Energy_Consumption_kWh'].values.reshape(-1
```

```
# Scale the data
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

# Function to create sequences for LSTM
def create_sequences(data, look_back=1):
    X, y = [], []
    for i in range(len(data) - look_back):
        X.append(data[i:(i + look_back), 0])
        y.append(data[i + look_back, 0])
    return np.array(X), np.array(y)

# For this very small dataset, a look_back of 1 is the most feasible.
# A larger look_back would leave too few samples for training.
look_back = 1
X, y = create_sequences(scaled_data, look_back)

# Reshape input to be [samples, time steps, features]
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

print(f"Original data points: {len(data)}")
print(f>Data points after creating sequences (X, y): {len(X)}")
print(f"Shape of X for LSTM: {X.shape}")
print(f"Shape of y for LSTM: {y.shape}")

print("Data prepared for LSTM model.")
```

```
Original data points: 4
Data points after creating sequences (X, y): 3
Shape of X for LSTM: (3, 1, 1)
Shape of y for LSTM: (3,)
Data prepared for LSTM model.
```

```
# Build the LSTM model
model = Sequential()
model.add(LSTM(units=50, activation='relu', input_shape=(look_back, 1)))
model.add(Dense(units=1))
model.compile(optimizer='adam', loss='mean_squared_error')

model.summary()

# Train the model
# Due to extremely limited data, epochs and batch_size are set low for demon
# In a real-world scenario with more data, these would be significantly high
print("\nTraining the LSTM model...")
model.fit(X, y, epochs=50, batch_size=1, verbose=0) # Set verbose to 0 to su
print("Model training complete.")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: User
super().__init__(**kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 50)	10,400
dense_1 (Dense)	(None, 1)	51

Total params: 10,451 (40.82 KB)

Trainable params: 10,451 (40.82 KB)

Non-trainable params: 0 (0.00 B)

Training the LSTM model...

Model training complete.

```
# Make predictions
# We will use the last available data point to predict the next future step
# Then use that prediction to predict the step after, and so on.

# First, predict on the existing data to see how well it fits
predicted_scaled = model.predict(X)
predicted_original = scaler.inverse_transform(predicted_scaled)

# For forecasting future values, we need to create an input sequence for the
# Given our data points are sparse (2011, 2016, 2021, 2026), predicting 2031
last_value_scaled = scaled_data[-1].reshape(1, look_back, 1)

future_predictions_scaled = []
num_future_steps = 2 # Predict for 2031 and 2036
current_input = last_value_scaled

for _ in range(num_future_steps):
    next_prediction_scaled = model.predict(current_input)
    future_predictions_scaled.append(next_prediction_scaled[0, 0])
    # Update input for the next prediction: the new input sequence includes
    current_input = np.append(current_input[:, 1:, :], next_prediction_scaled)
    # If look_back > 1, this needs adjustment to correctly slide the window.
    # For look_back=1, it's just the previous prediction.

future_predictions_original = scaler.inverse_transform(np.array(future_predi

# Prepare data for plotting
train_years = annual_total_energy['Year'].iloc[look_back:].values
train_actual = annual_total_energy['Total_Energy_Consumption_kWh'].iloc[look

# Create a full range of years including future predictions
all_years = list(annual_total_energy['Year'].values)
last_year = all_years[-1]
for i in range(1, num_future_steps + 1):
    all_years.append(last_year + (i * 5)) # Assuming 5-year intervals

# Combine actual and future predictions for plotting
```

```
plot_data_actual = annual_total_energy.copy()

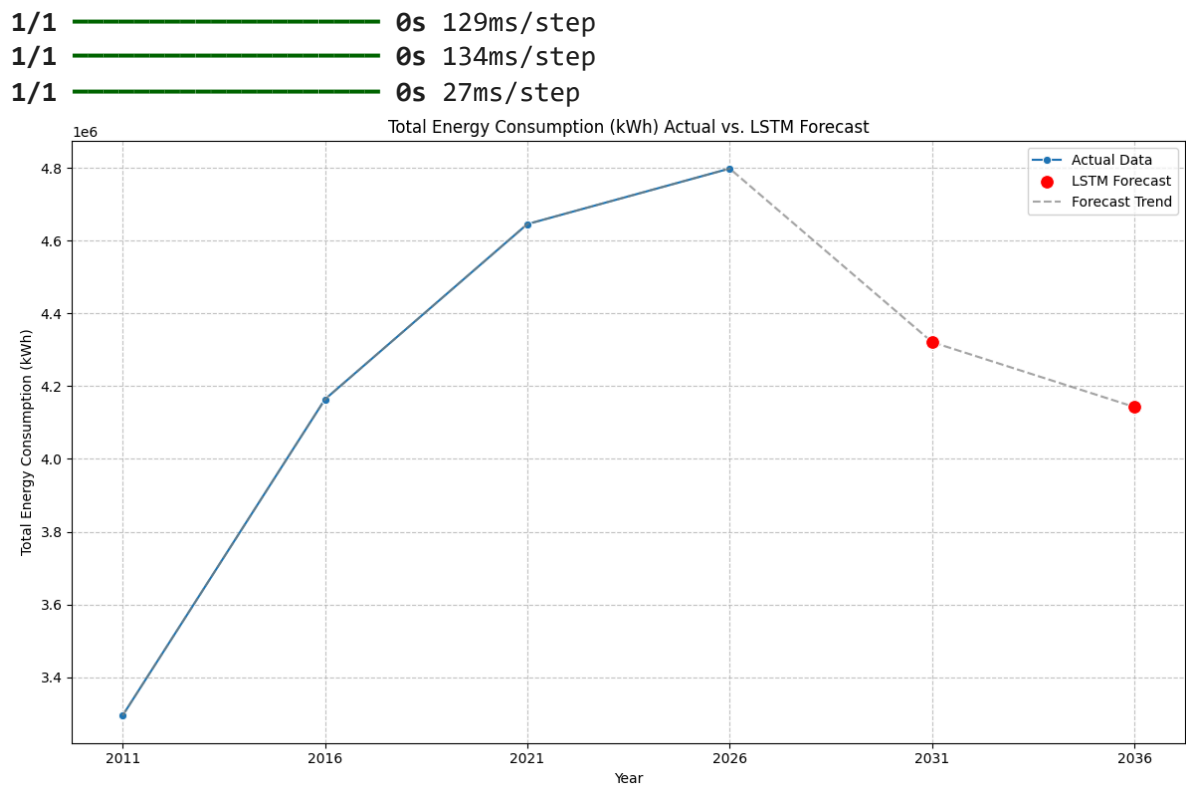
future_years = [last_year + (i * 5) for i in range(1, num_future_steps + 1)]
future_df = pd.DataFrame({
    'Year': future_years,
    'Total_Energy_Consumption_kWh': future_predictions_original.flatten()
})

forecast_df = pd.concat([plot_data_actual, future_df], ignore_index=True)

# Visualize the results
plt.figure(figsize=(12, 7))
sns.lineplot(x='Year', y='Total_Energy_Consumption_kWh', data=plot_data_actu
sns.scatterplot(x='Year', y='Total_Energy_Consumption_kWh', data=future_df,
plt.plot(forecast_df['Year'], forecast_df['Total_Energy_Consumption_kWh'], l

plt.title('Total Energy Consumption (kWh) Actual vs. LSTM Forecast')
plt.xlabel('Year')
plt.ylabel('Total Energy Consumption (kWh)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(all_years) # Ensure all years including future ones are shown
plt.legend()
plt.tight_layout()
plt.show()

print("\nLSTM Forecasting complete. Please review the plot and consider the
```

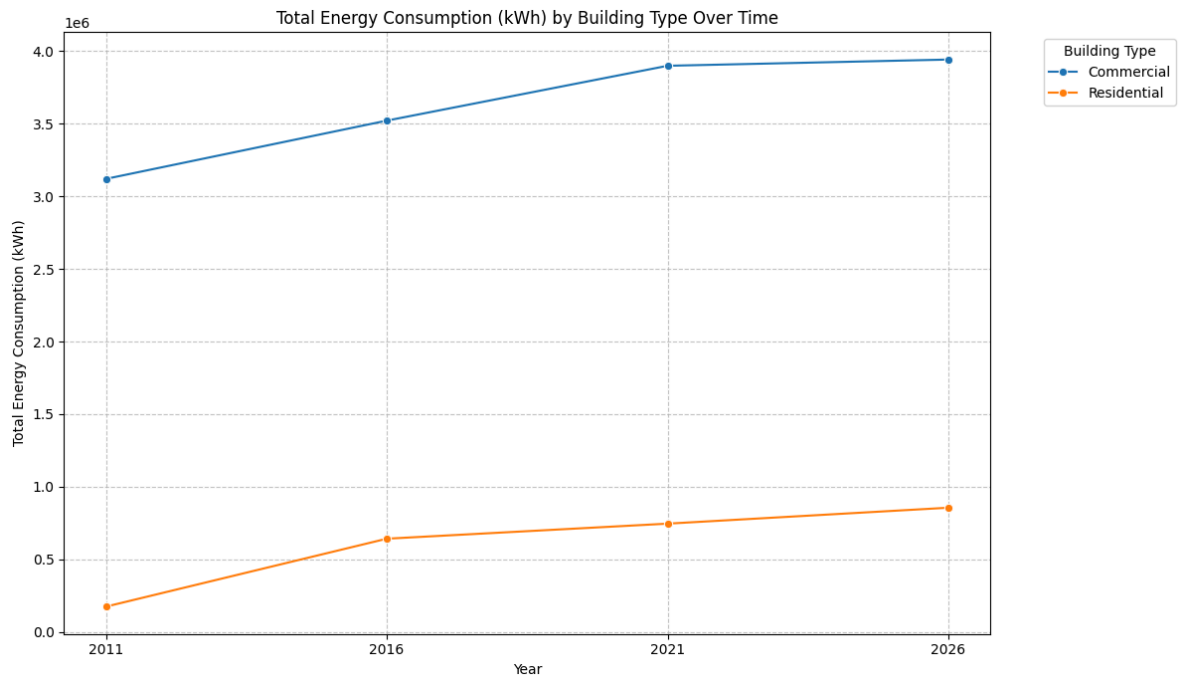
LSTM Forecasting complete. Please review the plot and consider the data limit

```
import matplotlib.pyplot as plt
import seaborn as sns

# Group by Year and Building Type and sum the total energy consumption
energy_by_building_type = df_energy_long_typed.groupby(['Year', 'Building Ty

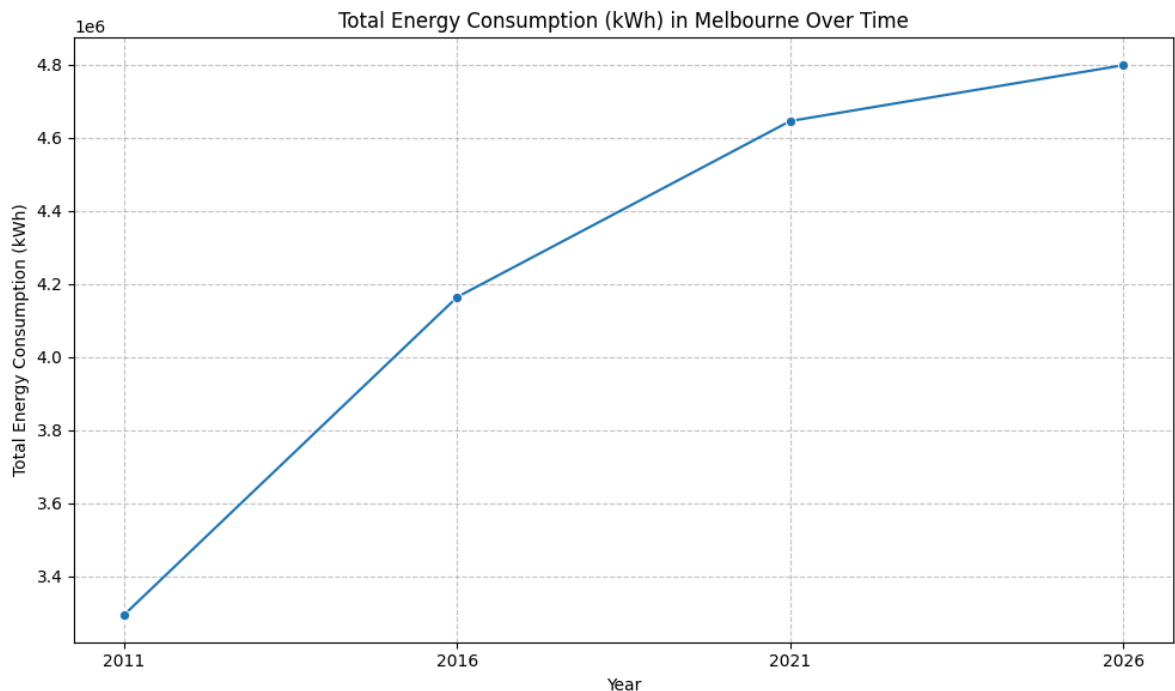
plt.figure(figsize=(12, 7))
sns.lineplot(x='Year', y='Energy_Consumption_kWh', hue='Building Type', data
plt.title('Total Energy Consumption (kWh) by Building Type Over Time')
plt.xlabel('Year')
plt.ylabel('Total Energy Consumption (kWh)')
plt.grid(True, linestyle='--', alpha=0.7)
```

```
plt.xticks(energy_by_building_type['Year'].unique())  
plt.legend(title='Building Type', bbox_to_anchor=(1.05, 1), loc='upper left')  
plt.tight_layout()  
plt.show()
```



```
# Group by year and sum the total energy consumption
annual_total_energy = df_energy_long.groupby('Year')['Total_Energy_Consumpti

plt.figure(figsize=(10, 6))
sns.lineplot(x='Year', y='Total_Energy_Consumption_kWh', data=annual_total_e
plt.title('Total Energy Consumption (kWh) in Melbourne Over Time')
plt.xlabel('Year')
plt.ylabel('Total Energy Consumption (kWh)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(annual_total_energy['Year'].unique())
plt.tight_layout()
plt.show()
```



```
!pip install tqdm
```

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-package

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Get the processed dataframe for energy consumption
df_energy_processed = processed_dataframes['df_energy_consumption_processed']

print("Columns in df_energy_processed:")
print(df_energy_processed.columns)

# Identify columns that represent energy consumption for different years
year_columns = [col for col in df_energy_processed.columns if 'total_' in col]

# Reshape the dataframe using `melt` to convert year columns into rows
# Now include 'Location' along with 'Geo Point' as id_vars
df_energy_long = df_energy_processed.melt(id_vars=['Geo Point', 'Location'],
                                           value_vars=year_columns,
                                           var_name='Year_Str',
                                           value_name='Total_Energy_Consumption')

# Extract the year from the 'Year_Str' column and convert to datetime
df_energy_long['Year'] = df_energy_long['Year_Str'].str.replace('total_', '')
df_energy_long['Date'] = pd.to_datetime(df_energy_long['Year'], format='%Y')

# Drop the original 'Year_Str' column as we now have 'Year' and 'Date'
df_energy_long = df_energy_long.drop(columns=['Year_Str'])

print("Reshaped df_energy_consumption_processed into a time-series friendly")
display(df_energy_long.head())
print(df_energy_long.info())
```

```
Columns in df_energy_processed:
Index(['Geo Point', 'Suburb', 'Latitude', 'Longitude', 'Geo Shape',
      'total_2011', 'p_com_2011', 'p_res_2026', 'p_com_2026', 'total_2026',
      'p_com_2021', 'property_id', 'p_res_2011', 'total_2016', 'total_2021',
      'p_res_2021', 'floor_area', 'p_res_2016', 'p_com_2016', 'Location'],
      dtype='object')
Reshaped df_energy_consumption_processed into a time-series friendly format,
```

	Geo Point	Location	Total_Energy_Consumption_kWh	Year	Date
0	-37.83363154012436, 144.98420174077557	Richmond	15.836556	2011	2011-01-01
1	-37.79900382289352, 144.93494112057527	West Melbourne	338.761383	2011	2011-01-01
2	-37.800632838889626, 144.9488946131469	North Melbourne	19.007585	2011	2011-01-01
3	-37.801031454537835, 144.95223929995998	North Melbourne	2.376923	2011	2011-01-01
4	-37.8004776165655, 144.94154189092782	West Melbourne	531.058950	2011	2011-01-01

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 54348 entries, 0 to 54347
Data columns (total 5 columns):
```

```
energy_by_building_type_location = df_energy_long_typed.groupby(['Year', 'Bu

plt.figure(figsize=(16, 9))
sns.lineplot(x='Year', y='Energy_Consumption_kWh', hue='Building Type', styl
plt.title('Total Energy Consumption (kWh) by Building Type and Location Over
plt.xlabel('Year')
plt.ylabel('Total Energy Consumption (kWh)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(energy_by_building_type_location['Year'].unique())
plt.legend(title='Building Type / Location', bbox_to_anchor=(1.05, 1), loc='
plt.tight_layout()
plt.show()
```

